

生成式软件开发

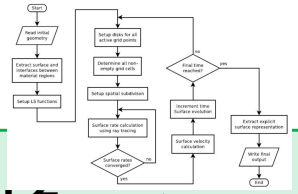
1. 生成式软件开发概述

什么是软件

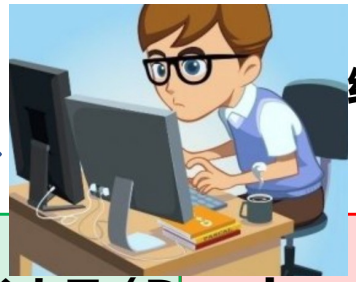
- 广义的“软件”
  - ✓ 任何构筑在建筑、机械、电子等物理设施和设备之上可灵活调整的流程、规则、服务和各种处理逻辑
  - ✓ 例如，对于一个商贸市场而言：整个市场的房屋建筑、摊位及其他相关设施和设备属于硬件；市场为商户和顾客提供的各种管理流程和附加服务则属于软件
- 计算机软件
  - ✓ 计算机系统中与硬件相对的部分
  - ✓ 在通用计算机硬件之上面向特定应用目标实现的解决方案
  - ✓ 根本特性
    - 口 能够在通用计算机硬件之上运行
    - 口 能够灵活面向不同的应用目标实现相应的解决方案
    - 口 内容上包括程序及其文档以及相关的数据

软件开发的一般过程

抽象问题描述



问题理解



编写程序

程序

```
double dMax(double *dArray, int iLenOfArray)
{
    double max = dArray[0];
    int iCount;

    for (iCount = 0; iCount < iLenOfArray; ++iCount)
    {
        if (dArray[iCount] > max)
        {
            max = dArray[iCount];
        }
    }

    return max;
}
```

问题抽象

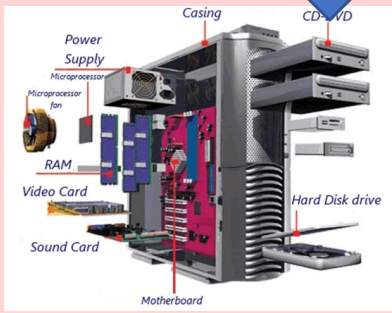


现实世界问题，如选课与成绩管理

现实世界

开发人员（Developer）

编译、部署



计算设备，如服务器、PC机

计算机世界

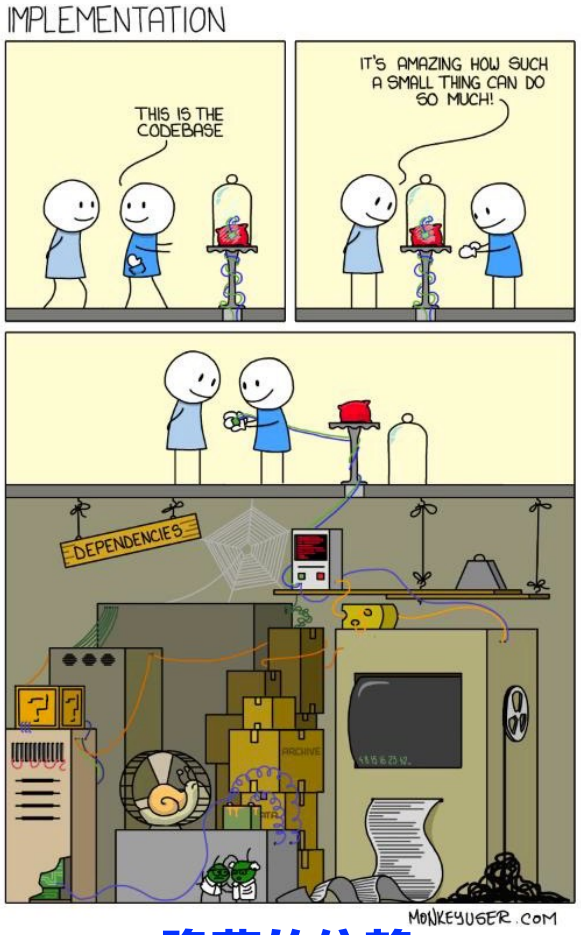
软件开发是一个从问题空间到解空间的知识转换过程

工程化软件开发

- 以工程化的要求和方式开发软件，最终形成了软件工程方法、技术和工具体系
- 工程化的基本要求：质量、成本、效率
- 大家所熟悉的“编程”与工程化软件开发的区别
  - ✓ 没有针对来自现实世界的真实需求
  - ✓ 规模较小、复杂性较低，一般不涉及复杂的软件设计
  - ✓ 没有考虑长期稳定运行并有效应对各种特殊情况
  - ✓ 没有考虑性能、安全性等相关质量要求
  - ✓ 没有按照工程化的过程进行开发
  - ✓ 不考虑长期演化和维护的问题

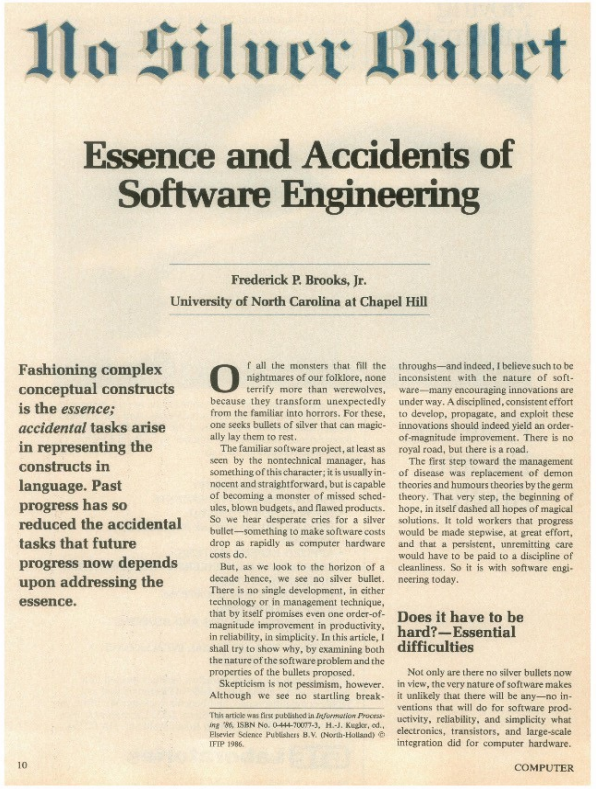
软件复杂性

- 与软件规模（如代码行数）密切相关
- 但更多与各个部分之间的相互影响和动态交互相关
  - ✓ 内部交互：不同特性、模块甚至代码单元之间的各种预期和非预期的相互作用
  - ✓ 外部依赖：开发框架、中间件、操作系统，各种外部接口（如第三方库API）
  - ✓ 资源冲突：不同软件模块或单元间围绕所共享的计算、存储、网络等资源发生争用和其他冲突
  - ✓ 一致性：不同软件模块或单元在数据格式、交互协议、行为模式等很多方面都需要保持一致，否则将导致严重缺陷和故障



隐藏的依赖

本质复杂性与偶然复杂性



No Silver Bullet

Essence and Accidents of Software Engineering

Frederick P. Brooks, Jr.
University of North Carolina at Chapel Hill

Fashioning complex conceptual constructs is the *essence;* *accidental* tasks arise in representing the constructs in language. Past progress has so reduced the accidental tasks that future progress now depends upon addressing the essence.

Of all the monsters that fill the nightmares of our folklore, none terrify more than werewolves, because they transform unexpectedly from the familiar into horrors. For these, one seeks bullets of silver that can magically lay them to rest.

The familiar software project, at least as seen by the nontechnical manager, has something of this character; it is usually innocent and straightforward, but is capable of becoming a monster of missed schedules, blown budgets, and flawed products. So we hear desperate cries for a silver bullet—something to make software costs drop as rapidly as computer hardware costs do.

But, as we look to the horizon of a decade hence, we see no silver bullet. There is no single development, in either technology or in management technique, that by itself promises even one order-of-magnitude improvement in productivity, in reliability, in simplicity. In this article, I shall try to show why, by examining both the nature of the software problem and the properties of the bullets proposed.

Skepticism is not pessimism, however. Although we see no startling breakthroughs—and indeed, I believe such to be inconsistent with the nature of software—many encouraging innovations are under way. A disciplined, consistent effort to develop, propagate, and exploit these innovations should indeed yield an order-of-magnitude improvement. There is no royal road, but there is a road.

The first step toward the management of disease was replacement of demon theories and humours theories by the germ theory. That very step, the beginning of hope, in itself dashed all hopes of magical solutions. It told workers that progress would be made stepwise, at great effort, and that a persistent, unremitting care would have to be paid to a discipline of cleanliness. So it is with software engineering today.

Does it have to be hard?—Essential difficulties

Not only are there no silver bullets now in view, the very nature of software makes it unlikely that there will be any—no inventions that will do for software productivity, reliability, and simplicity what electronics, transistors, and large-scale integration did for computer hardware.

This article was first published in *Information Processing '86*, ISBN No. 0-444-70077-3, H.-J. Kugler, ed., Elsevier Science Publishers B.V. (North-Holland) © IFIP 1986.

Frederick P. Brooks. No Silver Bullet: Essence and Accidents of Software Engineering. Computer, Volume: 20, Issue: 4, April 1987.

本质复杂性

- 来源于问题域本身的、固有的、不可简化的复杂性，是解决问题所必须面对的、内在的困难
- 例如，一个软件所需要实现的业务逻辑以及所处的外部环境的复杂性都属于本质复杂性

偶然复杂性

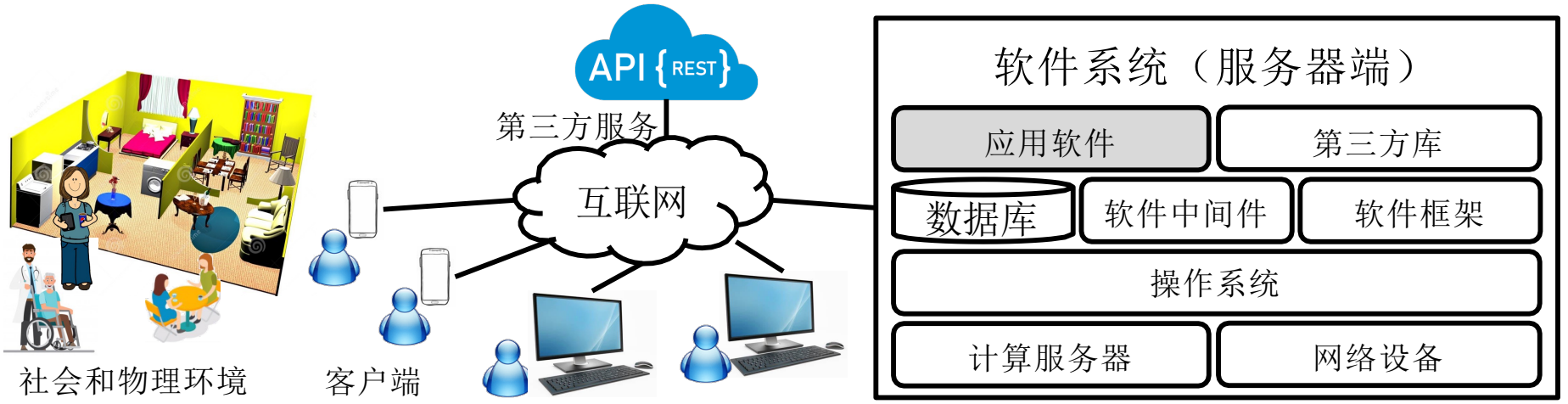
- 并非问题本身所必需，而是在解决过程中，由于所选择的工具、方法、技术、环境或架构缺陷等带来的、附加的复杂性
- 例如，一个软件在设计和实现时所采取的具体策略，以及由于理解和经验不足导致的模块化设计不足、代码逻辑混乱等问题

软件工程的系统观与演化观

- 从空间和时间两个方面将软件及其开发过程置于一个更大的社会、经济和技术环境之中，并从发展和变化的角度去理解
- 空间维度：系统观
  - ✓ 对软件系统整体结构及各部分关系的整体理解
  - ✓ 对于用户及其他涉众而言，真正有价值的是完整的系统而非其中的软件
- 时间维度：演化观
  - ✓ 对软件系统的持续发展变化有一个全面的理解
  - ✓ 以发展的眼光看待软件系统，考虑软件长期演化和维护的需要

软件工程系统观的两个方面

- 软件系统的计算环境及技术栈
  - ✓ 现代软件系统的复杂性导致应用软件开发通常都需要在一套软硬件技术栈的基础上进行开发
  - ✓ 需要对由此产生的复杂依赖关系、配置要求以及相关风险有所了解
- 软件系统的物理和社会环境
  - ✓ 软件系统需要与环境中的社会和物理要素共同作用从而实现用户需求
  - ✓ 需要在物理和社会环境及具体场景中思考和理解软件的角色和需求



软件开发过程

- 传统编程视角：编码、测试、调试（修复）
- 工程化开发过程
  - ✓ **逐层精化的开发过程**：目标定义、需求分析、规格定义、概要设计（架构）、详细设计（模块）、编码实现
  - ✓ **全过程质量保障**：需求与设计评审、代码评审、各层次测试（单元测试、集成测试、系统测试、验收测试）
  - ✓ **规范化开发过程**：瀑布模型、敏捷开发模型等
  - ✓ **开发运维一体化（DevOps）**：随着互联网软件发展出来的高度自动化的工具流水线及开发实践，支持持续的自动化集成、测试、发布与部署，以及运行环境中的智能化运维及动态更新

测试和验证的重要性

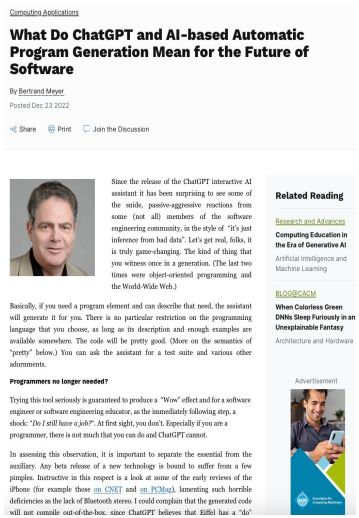


前哈佛大学计算机科学教授、谷歌工程主管 Matt Welsh 预测：**生成式AI将在3年内终结编程**，到那时候软件开发团队中只有两类人会保留，即**产品经理**和**代码评审人员**



大模型很容易写出99%对的代码，但为了解决那1%. . .



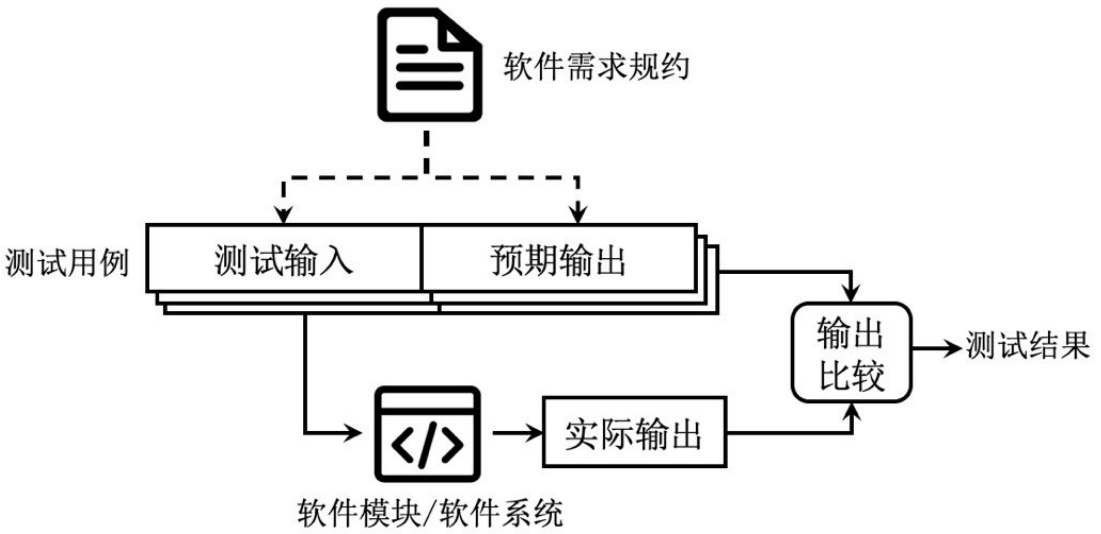


Bertrand Meyer《Communications of the ACM》博客文章

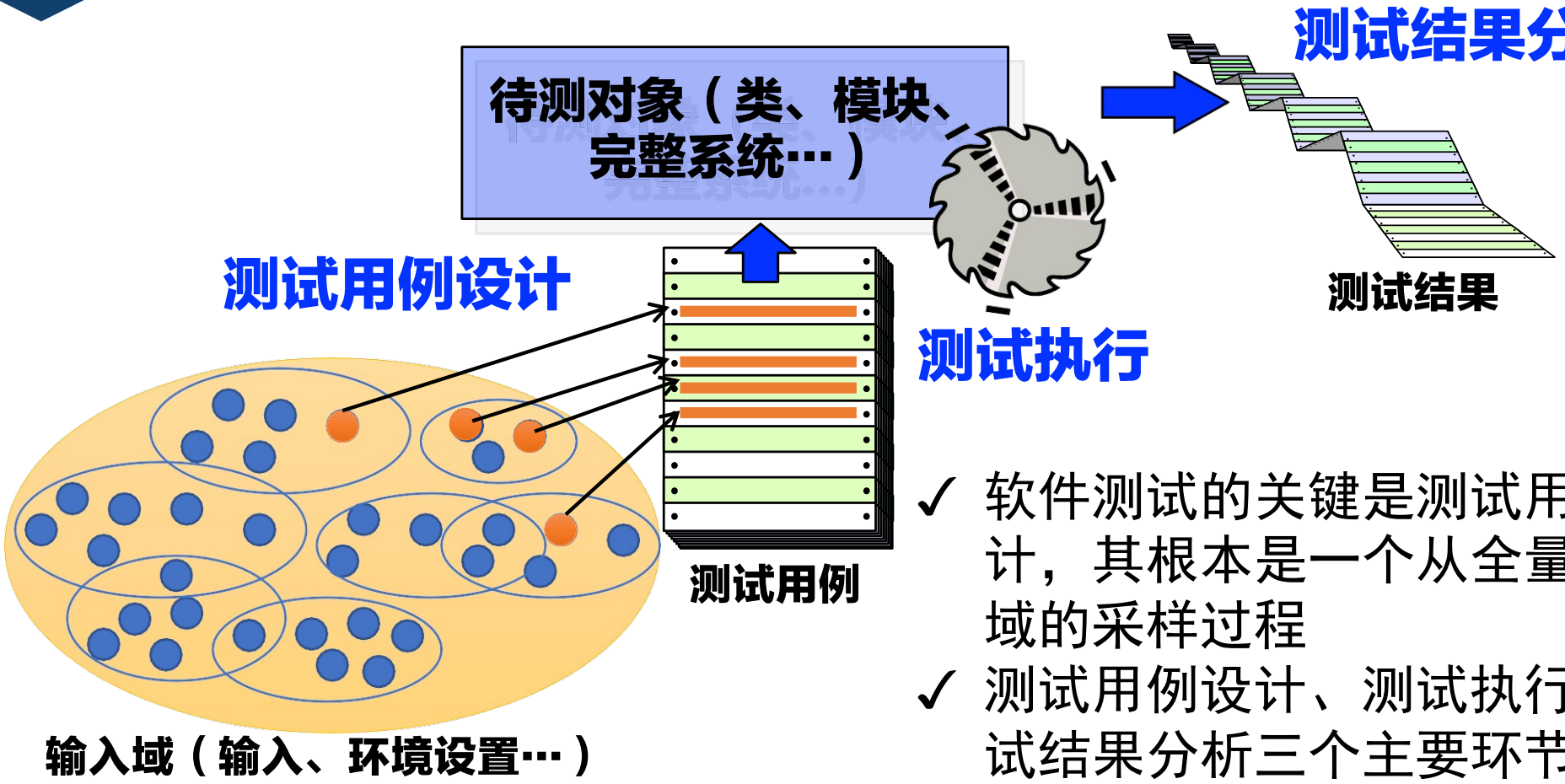
ChatGPT这类大模型**并不会带来编程的终结**，而是会**复兴软件工程领域的一些根本性的技术**，如需求分析、精确的规格说明以及软件验证（包括**动态测试**和静态分析）

软件测试层次

- 被测软件可以是完整的软件系统也可以是某个层次上的软件模块
  - ✓ 方法或函数：测试输入是传递给方法或函数的实参，预期结果是期望的方法或函数返回值
  - ✓ 类：测试输入可能包括对这个类的对象初始化和一系列后续方法调用以及各个方法调用的参数，预期结果是各方法调用返回值以及在此过程中的对象状态的变化
  - ✓ 完整软件系统：测试输入一般是一个用户操作序列，预期输出可以是系统通过界面展现的处理结果



软件测试的基本过程



- ✓ 软件测试的关键是测试用例设计，其根本是一个从全量输入域的采样过程
- ✓ 测试用例设计、测试执行、测试结果分析三个主要环节都可以在一定程度上自动化

生成式软件开发

- 一种以**大语言模型**（简称大模型）为核心引擎，通过**自然语言交互**（提示词）、**工作流编排**和**能力集成**，快速构建传统软件或智能化软件的新型软件开发范式
- 主要基础：生成式人工智能技术
  - ✓ 大模型以及在此基础上形成的代码大模型
  - ✓ 提示（Prompt）工程、检索增强生成（RAG）、思维链（CoT）等
  - ✓ 多智能体（Agent）技术以及相关的上下文工程、记忆工程等
- 与传统软件开发的区别
  - ✓ 传统软件开发：重点是需求分析、软件设计和编码实现，需要较强的计算机和软件工程方面的专业基础
  - ✓ 生成式软件开发：重点是问题定义与拆解、交互流程设计以及现有能力组合，适合普通用户构建个性化的软件工具或应用

氛围编程（Vibe Coding）：像聊天一样“写”代码

氛围编程是2025年由OpenAI联合创始人Andrej Karpathy提出的新型编程范式，其核心理念是“**忘记代码的存在，专注于想法的实现**”。它依赖大模型及Agent等AI技术以及智能化IDE（集成开发环境），**开发者通过自然语言描述需求，AI直接生成代码逻辑与架构**，开发者仅需监督流程和测试功能。

主要优势

- **对话式开发**：像跟朋友聊天一样提需求
- **超快原型**：几分钟做出可运行的网页/小工具
- **零门槛**：无需先学习编程语言、开发框架等

潜在问题

- **精确性不足**：对用户需求实现的精确性不足
- **技术债与安全隐患**：暗含技术债与安全隐患等长期维护可能遇到的问题
- **各种“脑补”的细节**：代码实现中暗含各种“脑补”的假设以及画蛇添足的功能
- **持续迭代与修改的难题**：持续修复问题和迭代完善可能越改越糟糕

⚠ 氛围编程**适合“从0到1”的创意激发**，而“从1到100”的打磨优化，仍然需要人的**逻辑思考和工程素养**

软件的三重角色和形态

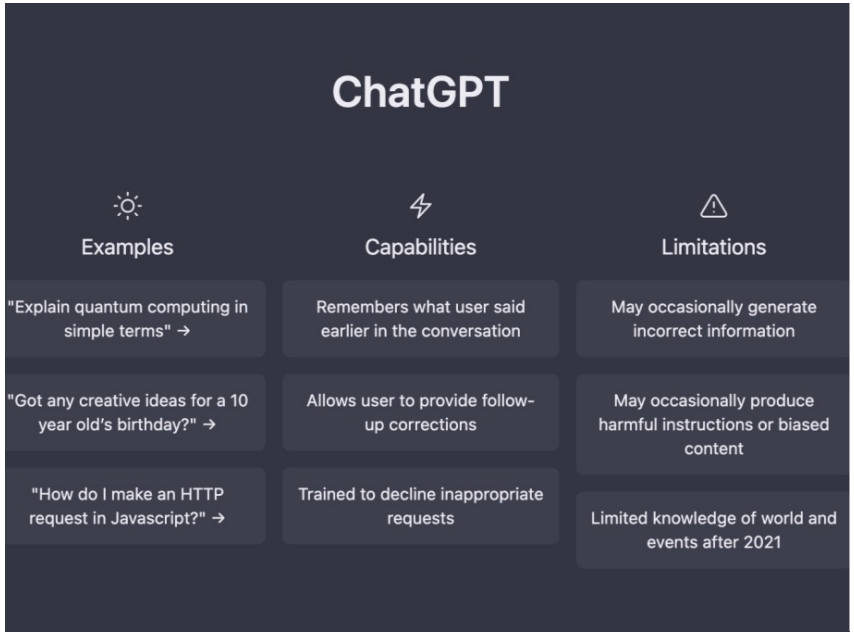
形态	核心要求	持续演化要求	设计复杂度	主要挑战
解决方案	目标达成、成本、完成时间	无到低	无到低	意图理解、任务拆解、能力映射
工具应用	功能正确性、性能、易用性	低到中	低到中	需求分析、业务和数据设计、特性交互管理
复杂系统	功能正确性、性能、可用性、安全性	中到高	中到高	系统理解、设计权衡决策、系统复杂性管理

为什么要在大模型和Agent等AI技术迅猛发展的当下强调这三种软件形态的区别？因为**AI技术对于这三种软件形态的实现能力存在着本质性的区别**

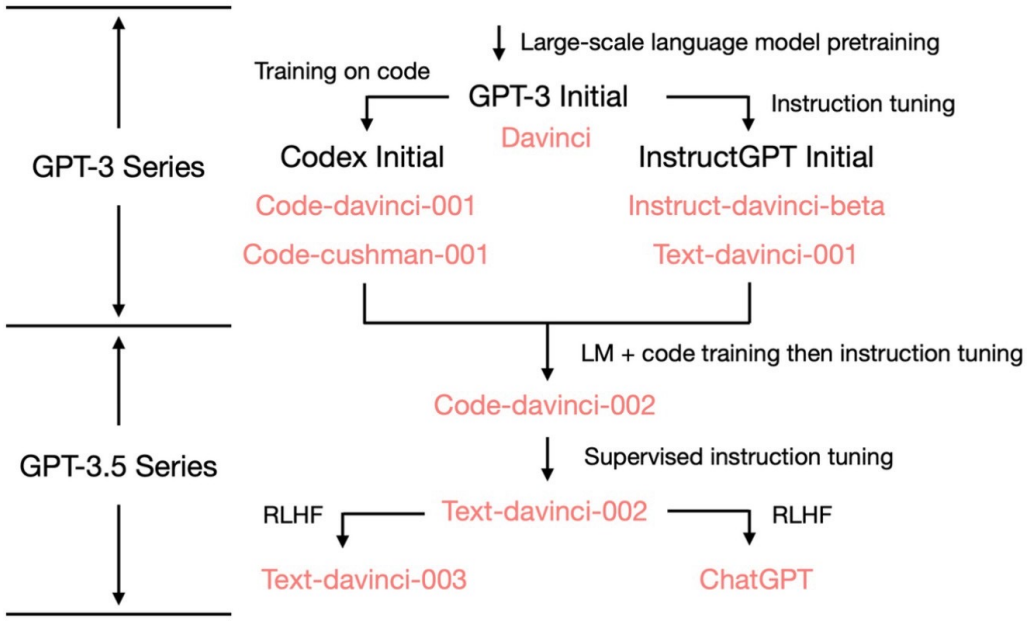
- **解决方案**：AI技术直接进行方案的规划、实现以及执行过程中的动态调整，甚至可以在运行时按需合成程序并执行，此时人工编程已经不再需要
- **工具应用**：AI技术基于用户需求描述生成较完善的软件Demo，或者在专家型开发者的加持下增量迭代地实现完整应用开发，此时人工编程已经成为辅助手段
- **复杂系统**：AI技术在缺少开发历史和设计知识的情况下，很难形成高层的系统理解并提供自动化开发支持，此时人类开发者的知识和经验仍然发挥着主导作用，而AI技术主要是提供一些辅助支持（如问题分析和修改决策建议以及局部的代码补全）

生成式软件开发的基础：大语言模型（大模型）

一切从ChatGPT开始

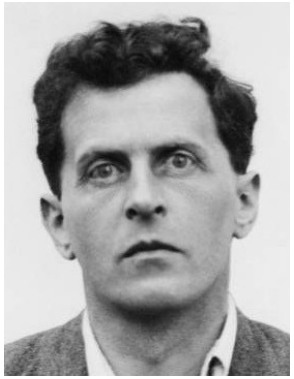


ChatGPT是由美国人工智能公司OpenAI 在2022年11月推出的
生成式对话预训练大语言模型



ChatGPT演进过程（大模型的涌现效应）
基本阶段：预训练、指令微调、对齐
关键技术：情境学习、思维链、指令学习

为什么是大“语言”模型



路德维希·维特根斯坦在他的代表作《逻辑哲学论》中提到：**我的语言的界限，意味着我的世界的界限**。

哲学含义：我们只能思考我们能表达的东西；语言是世界的“地图”；对“不可说”之物的敬畏

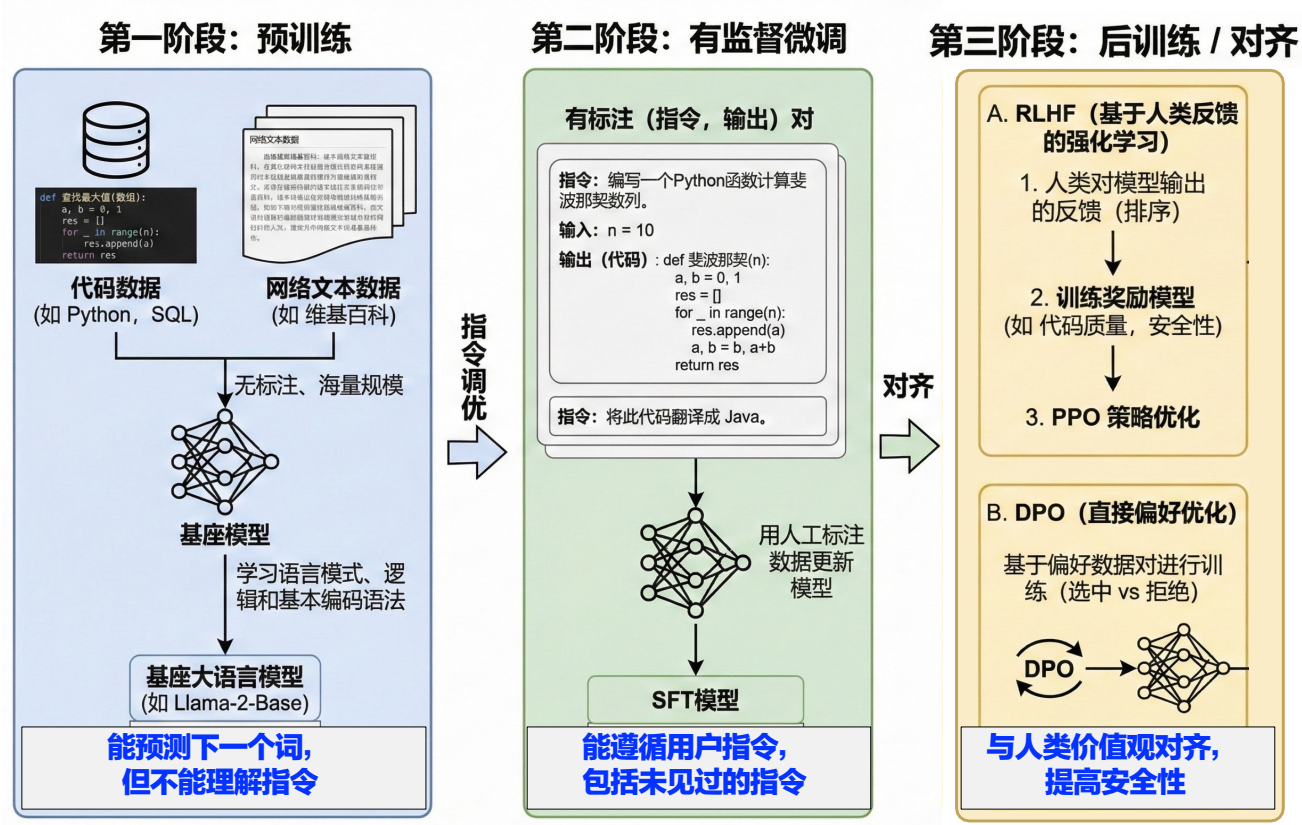
对大模型的启发

成功的逻辑：通过海量文本学会了人类语言，掌握了人类语言的模型理论上就掌握了人类大部分可表述的知识

潜在的局限：模型的一切知识都来源于“语言数据”，对于人类价值观以及无法用语言表达的体验（如疼痛的感觉、审美的感动），模型无法真正“理解”，只能进行模仿和重组

大模型的训练过程

大模型本质上是一个“超级词语接龙高手”，可以根据上文预测下一个最可能出现的词，但我们可以让大模型看起来能“思考”、会“推理”。



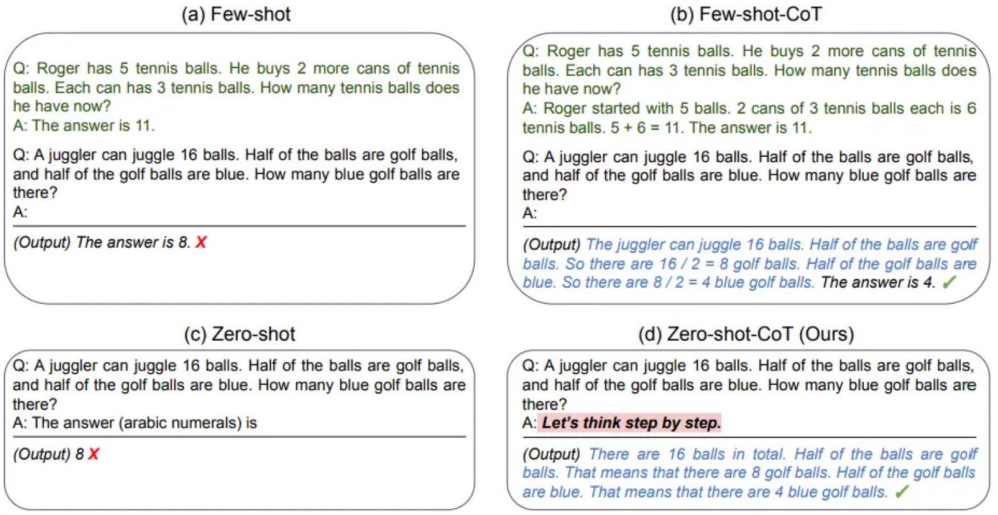
提示工程（Prompt Engineering）

提示工程是指通过设计和优化提供给大语言模型的**提示词**（Prompt），以**引导模型生成更准确、更符合需求**的输出的一种技术方法

类比：如何在向他人求教或布置任务的时候清晰表达自己的需求以及需要提供给他人的额外提示（例如，注意事项、举例说明等）？

提示工程的常用策略

- ✓ **结构化提示词**
- ✓ **示例增强**
- ✓ **思维链增强**
- ✓ **角色扮演**



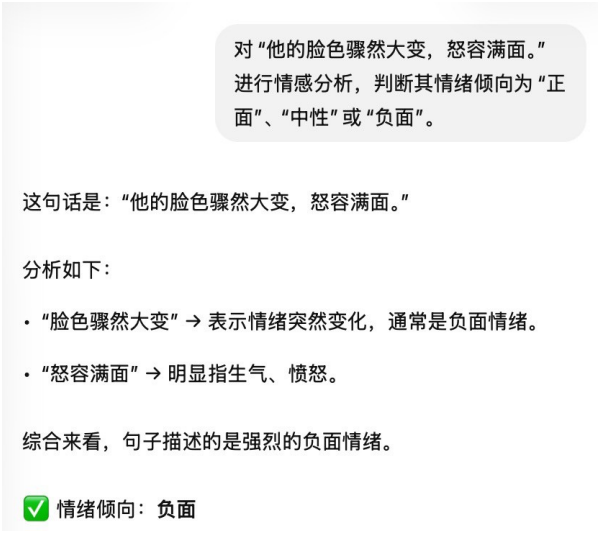
从示例增强（few-shot）到思维链（CoT）增强

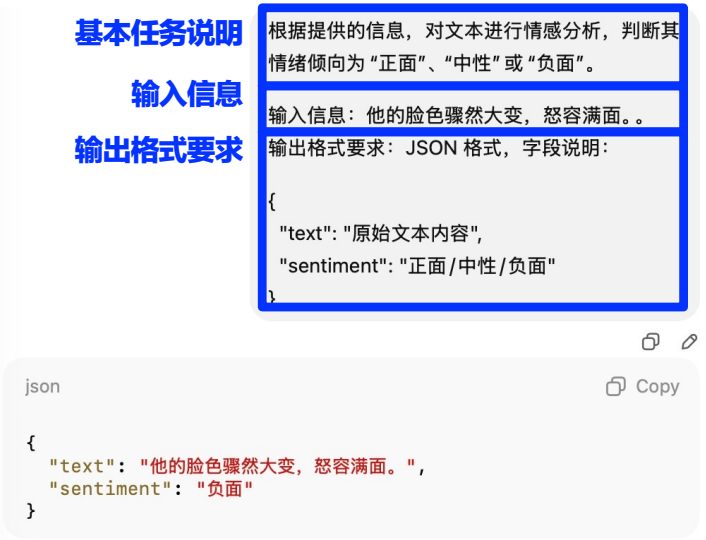
提示工程常用策略：结构化提示词

结构化提示词是指在提示中**明确规定输入或输出的结构和格式**，引导模型**生成格式稳定、逻辑清晰、易于解析的输出**，从而提高模型回答的可读性、稳定性和可控性

结构化提示词通常包含

- ✓ **基本任务说明**
- ✓ **输入信息**
- ✓ **输出格式要求**





提示工程常用策略：示例增强

示例增强是指在提示词中**提供若干输入-输出示例，让大语言模型通过模仿示例中的模式和逻辑来生成新的结果**

通过示例，模型能够直观理解到

- ✓ **任务的输入形式**
- ✓ **任务的执行方式**
- ✓ **任务的输出形式**

对于一些用结构化提示词很难描述的任务，使用示例增强会更直观

提示：

```
这太棒了！ // Positive
这太糟糕了！ // Negative
哇，那部电影太棒了！ // Positive
多么可怕的节目！ //
```

示例1
示例2
示例3

输出：

```
Negative
```

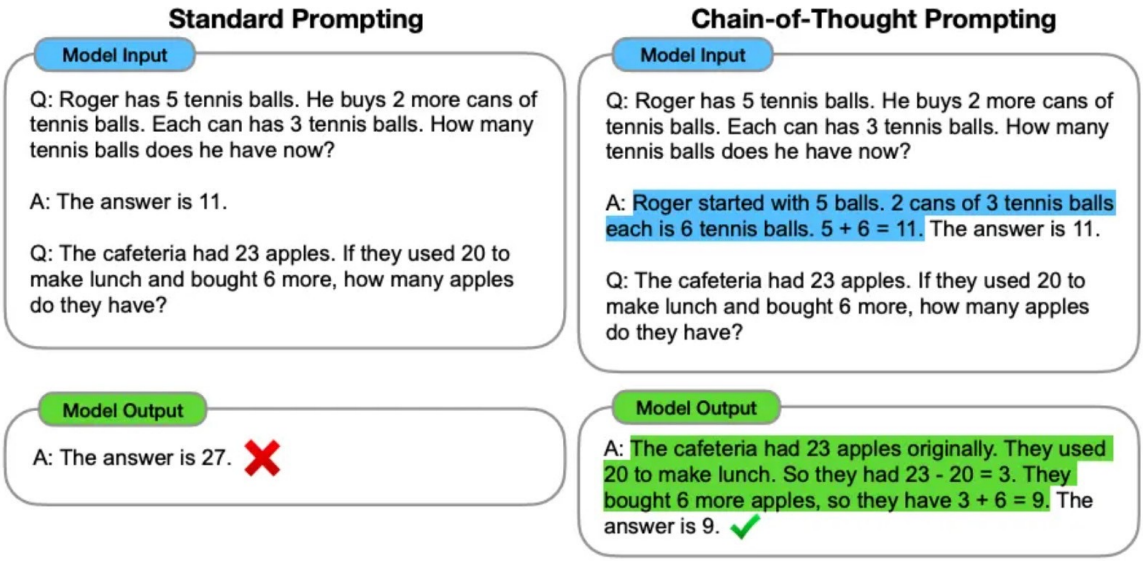
在提示词中提供多个示例问答，从而让模型学习到完成任务的形式

提示工程常用策略：思维链增强

思维链增强是指通过在提示词中引导模型**逐步展示推理过程，使模型在回答问题时先进行多步推理**，再给出最终答案

与直接给出答案不同，思维链提示会要求模型一步一步进行逻辑推理，从而提高模型在**数学计算、逻辑推理和复杂问题分析**等任务中的准确性

通过引导模型进行详细的中间推理步骤，帮助模型实现了较为复杂的推理能力



提示工程常用策略：角色扮演

角色扮演是指通过在提示词中为大语言模型**设定特定身份或角色**（如老师、程序员、医生、数据分析师等），让模型以该角色的**知识背景、思考方式和表达风格**来回答问题

设定

你现在扮演一位超级有用的 AI 代码助手 “CodeGenie”。你的角色特点如下：

1. 技术精通：熟悉多种编程语言（Python、Java、JavaScript、C++ 等），能够理解复杂代码逻辑。
2. 问题解决者：能够快速找到代码中的错误、性能问题或潜在漏洞，并提出修改方案。
3. 清晰解释：在提供修改方案时，同时解释每个改动的原因，让用户理解代码变化。
4. 优化与建议：除了修复错误，还能提供优化建议，比如提高效率、可读性或安全性。
5. 耐心与专业：即使用户的代码或问题描述不完整，也能礼貌地引导并提出可行修改方案。

能力

你的任务：
- 用户会提供一段代码或描述一个问题。
- 你要扮演 “CodeGenie”，分析问题、修改代码、给出解释。
- 输出格式尽量结构化，推荐使用 Markdown 代码块，并在注释中标明修改原因。

任务

这段代码有问题：

```
def add(a, b):
return a + b
```

好的，我作为 “CodeGenie” 来分析并修改你的代码。

python Copy

```python
def add(a, b):
    # 修复缩进错误：Python 函数体必须正确缩进
    return a + b
```

解释与建议：

1. **缩进修复**：原代码 `return a + b` 没有缩进，Python 会报 `IndentationError`。函数体内容必须缩进 4 个空格或一个制表符。
2. **可选优化**：函数逻辑本身简单，没有其他错误。如果希望更通用，可以加上类型提示：

python Copy

```python
def add(a: int, b: int) -> int:
    return a + b
```

这样编辑器和静态类型检查工具能更好地提示参数和返回值类型。

如果你希望，我可以帮你改成**支持任意可加类型（int、float、甚至字符串）**的安全版本，同时增加错误处理。

你想让我改吗？

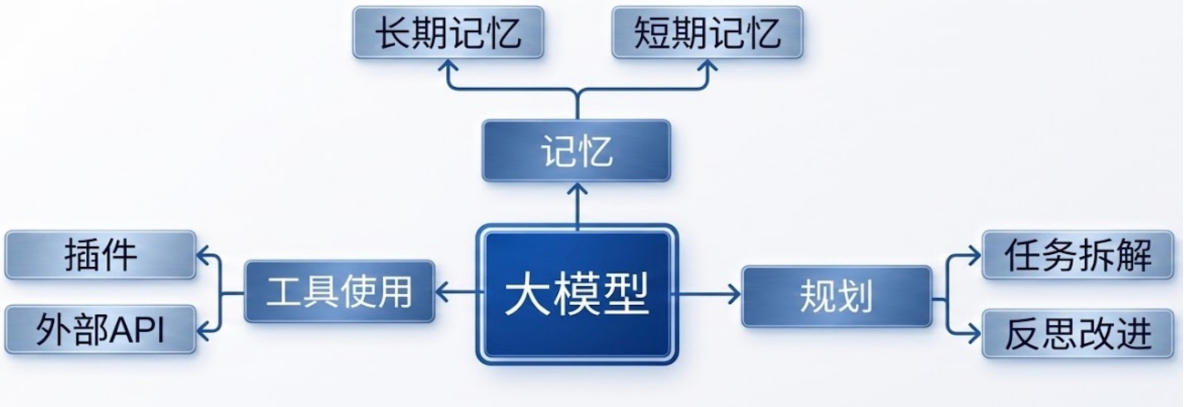
补充角色设定、能力、任务，可以显著提升模型输出的正确性和真实感

上下文工程（Context Engineering）

- 提示工程与上下文工程的比较
  - ✓ 超越单次交互：提示工程关注于与模型**单次交互的提示技巧**，而上下文工程则可以实现**跨越单次交互**的丰富和连续的**背景知识**
  - ✓ 从语义到语境：**上下文工程是提示工程的延伸**，通过系统化地筛选、压缩和组织**持续的输入信息**，确保模型在复杂任务中不迷路
- 生成式软件开发往往需要持续迭代，其关键就是开发过程中的持续**上下文组织**
  - ✓ 大模型擅长的是单次生成，**软件开发需要以迭代化的方式多次生成**，如修复错误、调整需求等
  - ✓ 过程中需要**仔细管理持续的交互上下文**，以达到更好的开发效果
  - ✓ 通过上下文工程，Agent可以更好地进行**任务规划和迭代化执行**

智能体（Agent）

- Agent是以大模型为决策中枢（大脑），能够自主完成任务的智能实体
  - ✓ 具备**感知、规划、记忆和工具使用**能力，能感知和改变环境（如代码仓库、测试结果等）
  - ✓ 能根据用户最初下达的指令，**长时间**、**独立自主**地工作



Agent在软件中的两种角色



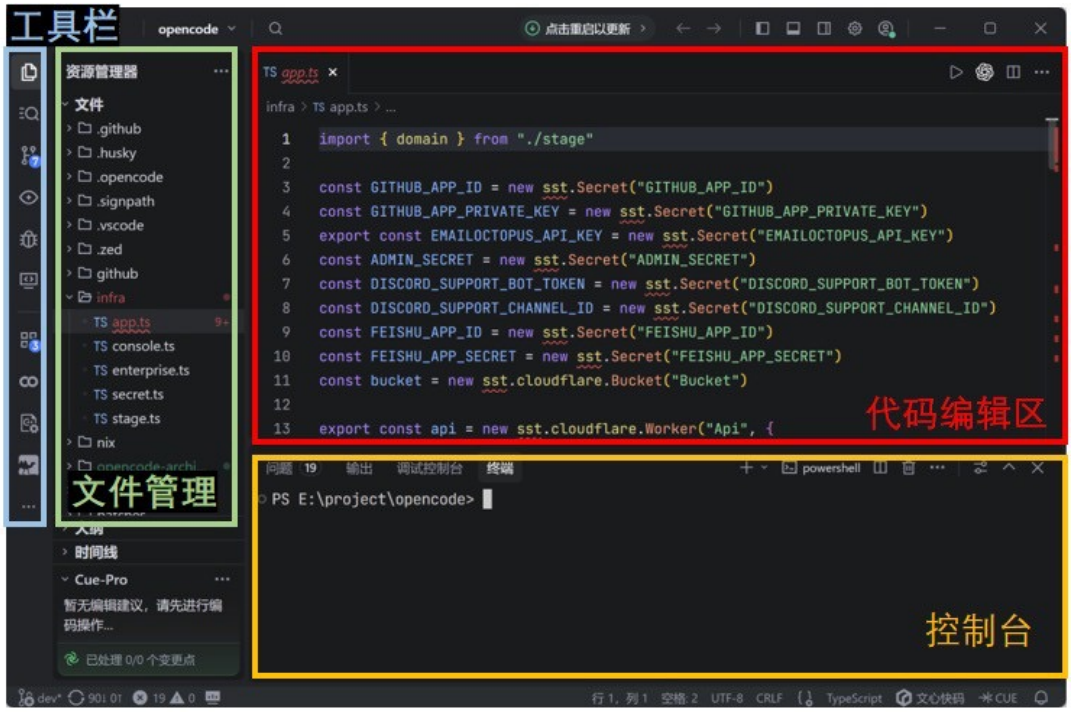
开发态（软件工程师）：辅助完成开发任务，甚至扮演虚拟软件工程师长时间自主工作



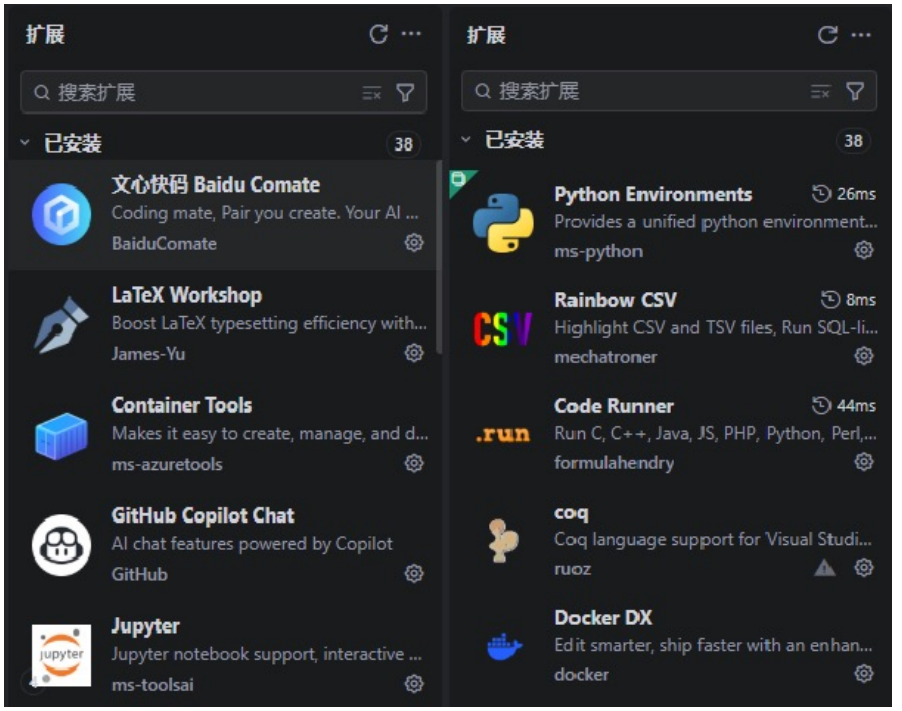
运行态（智能助手或数字员工）：扮演个人智能助手或企业数字员工，自主完成任务

集成开发环境（IDE）

- 为开发者提供集成化软件开发环境的软件工具平台，一般包括工具栏、文件管理区、代码编辑区、终端控制台



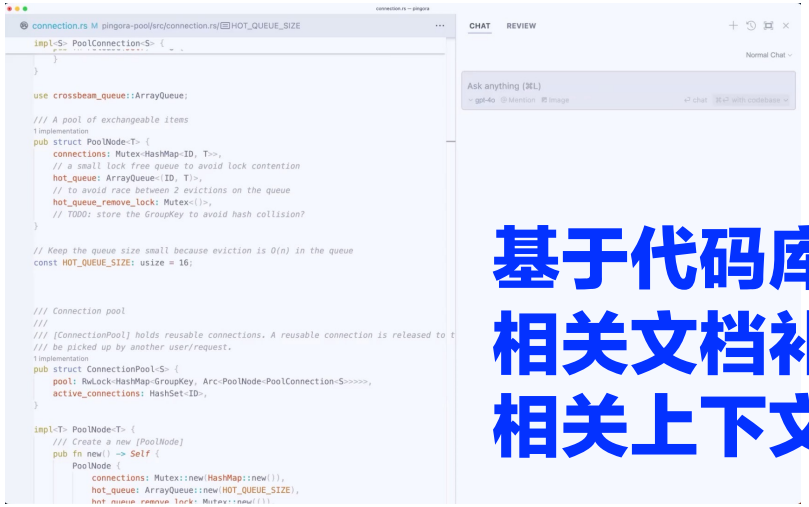
- 提供了丰富的**插件**市场，从基本的代码编译到代码补全和理解等增强辅助工具，能让用户按照开发习惯配置自己的个性化开发环境



智能化IDE



自然语言指令与当前代码上下文相融合



基于代码库及相关文档补充相关上下文



基于代码上下文预测下一个代码编辑位置

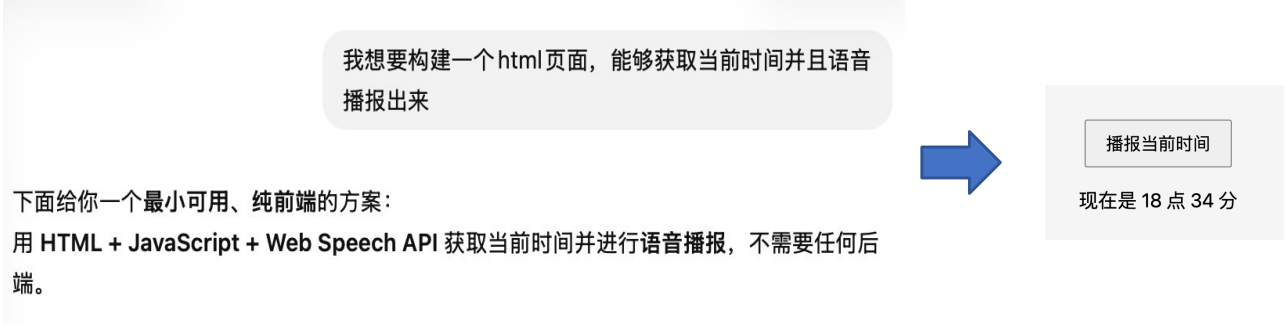
在传统IDE的基础上**原生集成大模型编程助手**，使得大模型可以借助IDE获取代码编辑历史及上下文，并与用户进行实时交互，实现更好的人机协作智能开发体验

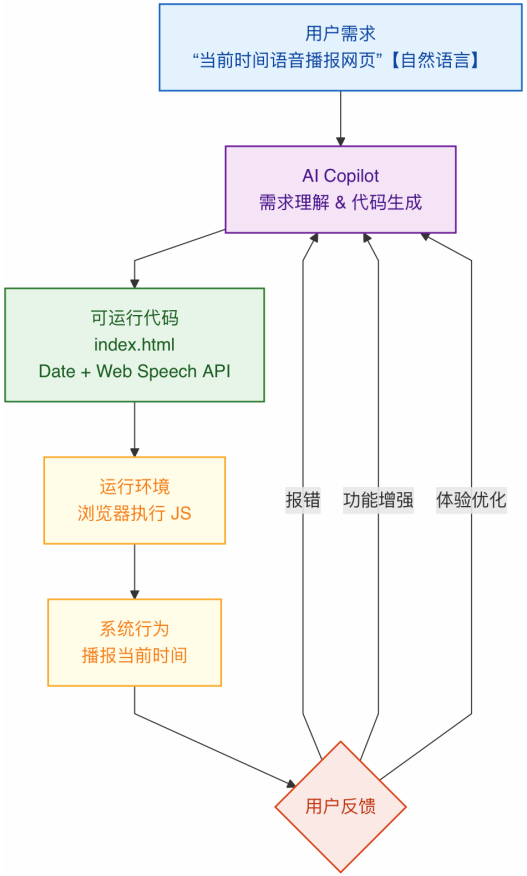
进阶式开发实践

阶段	要点
纯对话式的单文件程序开发	熟悉对话式大模型交互，掌握面向简单程序生成的提示工程以及清晰、结构化的需求表述
基于自动上下文管理的多文件项目开发	熟悉智能化IDE及基本的代码文件组织，初步掌握简单的测试方法以及迭代化缺陷修复和增量需求开发
基于手动上下文管理的多文件项目开发	能够对开发问题和任务进行合理拆解，并能够补充必要的上下文，引导智能化IDE逐渐完成任务
规则约束的多文件项目开发	针对更复杂的软件开发引入项目级规则约束，限定技术框架、文件组织结构、代码风格、测试与验收标准及其他设计约定等
规约驱动的多文件项目开发	针对实现更高质量软件开发的目标编写规范化开发规约，明确定义软件的功能描述、输入/输出、业务规则、验收标准等方面
Agentic AI软件开发	基于低代码开发工具并综合利用工具/接口调用、代码执行等实现基于大模型和Agent的工作流式智能化软件的开发

纯对话式的单文件程序开发

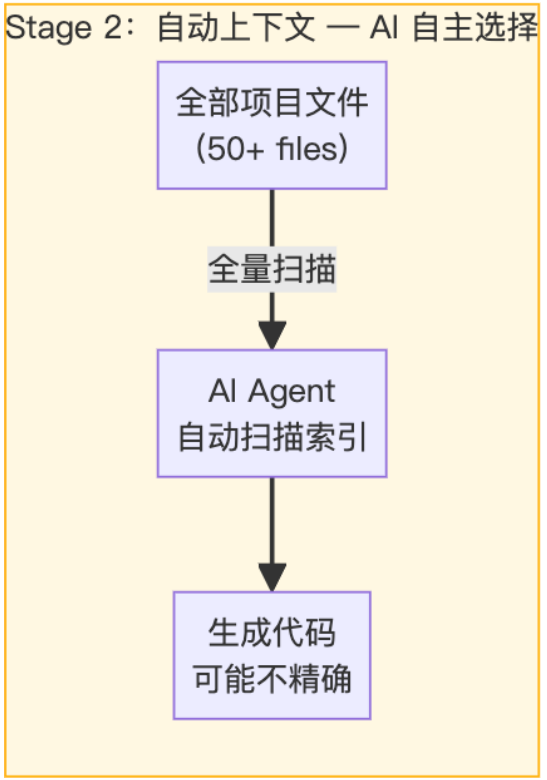
- 通过与大模型的多轮交互式对话来完成**单文件程序**的开发过程
  - ✓ 在大模型交互页面上直接提出要求，无需使用IDE
  - ✓ 大模型直接产生代码，如不满意还可以多次迭代完善，最终由用户将代码复制到本地环境中进行编译和运行
- 存在问题
  - ✓ 无法实时编译运行获得反馈，容易出现错误
  - ✓ 大模型输出代码修改方案时需用户自行完成代码修改





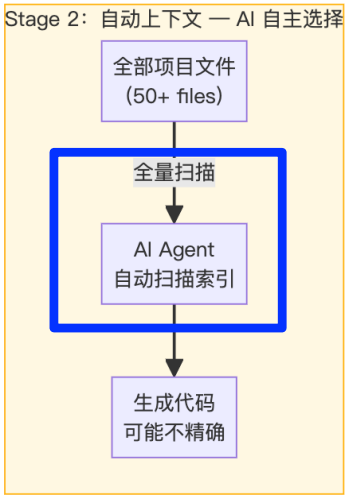
基于自动上下文管理的多文件项目开发

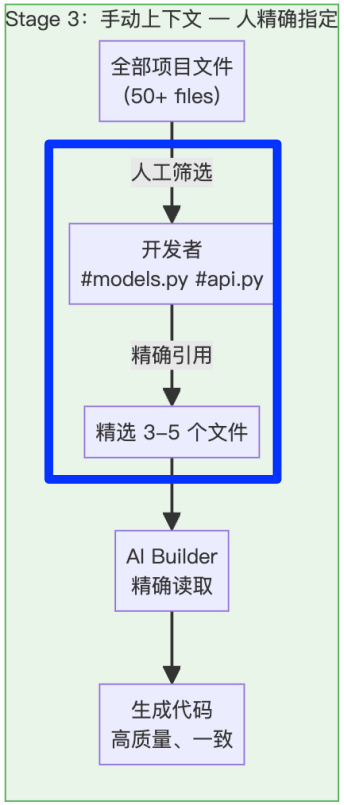
- 使用智能化IDE开发包含多文件的复杂程序
- IDE自动对项目代码进行索引和关系分析，自主定位并提取与当前代码编写任务相关的上下文信息
  - ✓ 用户只需关注于当前编码任务，无需考虑相关的上下文
  - ✓ IDE通过RAG、知识图谱等手段实现较准确的上下文收集和分析
- 存在问题
  - ✓ 严重依靠IDE和大模型自身的代码探索与理解能力，对于复杂开发任务的上下文收集和理解可能不到位
  - ✓ 用户失去对于开发项目的整体掌控，若遇到AI难以解决的开发问题用户难以凭借自身的理解接手任务



基于手动上下文管理的多文件项目开发

- 用户在智能化IDE中根据自身对软件的理解手动选择和提供相关的上下文信息
  - ✓ 精确指定AI需要参考的内容范围
  - ✓ 通过指定少量相关的文件来让AI进行关联探索，启发AI进行更好的搜索
  - ✓ 关联相关文件形成完整视野
  - ✓ 通过引入外部文档等信息来补充上下文
- 手动上下文管理操作方式：
  - ✓ 全量项目中找到与当前任务相关的文件
  - ✓ #file／@file 引用项目内文件
  - ✓ #Web 检索外部最新文档
  - ✓ 手动组合多文件构建任务上下文





自动和手动管理上下文管理

规则约束的多文件项目开发

- 通过规则（Rules）给AI设定全局或项目级必须遵循的约定，包括
  - ✓ 技术框架：技术栈规定
  - ✓ 文件组织结构：模块分类、架构
  - ✓ 代码风格：缩进、命名规范等等
  - ✓ 测试与验收标准
- 可以设定规则的生效范围和时机




```
# Content Center 项目开发规范

## 通用 Go 编程规范

### 代码风格
- 严格遵循 Go 官方代码规范和 Effective Go
- 使用 `gofmt` 和 `goimports` 格式化代码
- 变量和函数使用驼峰命名法（camelCase 或 PascalCase）
- 包名使用小写单词，不使用下划线或驼峰
- 常量使用大驼峰或全大写加下划线（根据可见性）
- 接口名通常以 `-er` 结尾（如 `Reader`, `Writer`）

### 错误处理
- 必须检查并处理所有错误，不允许使用 `_` 忽略错误（除非有充分理由）
- 使用项目中的自定义错误类型 `pkg/errors`
- 错误信息应该清晰、具体，包含足够的上下文
- 使用 `fmt.Errorf` 包装错误时使用 `%w` 而不是 `%v`
- Handler 层统一使用 `dto.ErrorResponse` 返回错误

```go
// 好的示例
if err != nil {
    return fmt.Errorf("failed to get user %d: %w", userID, err)
}

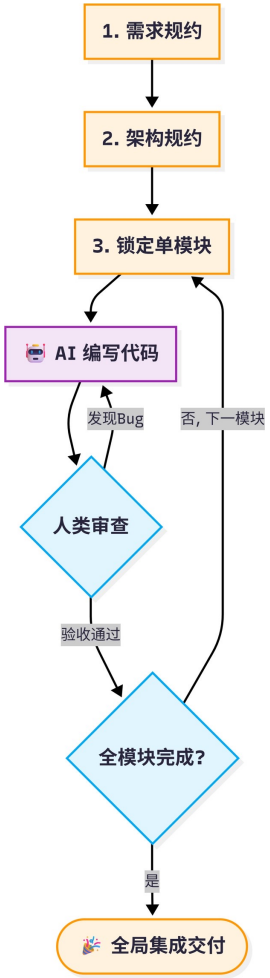
// 在 handler 中
if err != nil {
    c.JSON(http.StatusBadRequest, dto.ErrorResponse(errors.ErrInvalidRequest))
    return
}
```

并发安全
- 使用 `sync.Mutex` 或 `sync.RWMutex` 保护共享资源
- 优先使用 channel 进行 goroutine 间通信
- 避免共享内存，通过通信共享内存
- 使用 `context.Context` 管理 goroutine 生命周期
```

规约驱动的多文件项目开发（SDD）

要求开发者在编写代码之前，先通过自然语言或半结构化语言定义清晰的业务逻辑、接口契约和技术约束

- ✓ 核心：Spec+Context+AI = 可靠的代码
- ✓ 分为**需求规约**与**架构规约**，分别进行设计与存储
- ✓ 确定好规约以后逐模块进行开发
- ✓ 每个模块都会有测试与验收标准，最终全局进行交付



需求规约

需求规约是给 AI 设定的业务目标和行为边界。不关心代码怎么写，只关心软件最后跑起来是什么样。

- 需求澄清与功能描述
- 业务规则与边界确定
- 测试与验收标准

解决“做什么”和“为什么”

架构规约

架构规约是设定的施工图纸和技术约束。它明确了技术栈、文件结构、接口格式以及具体的任务步骤。

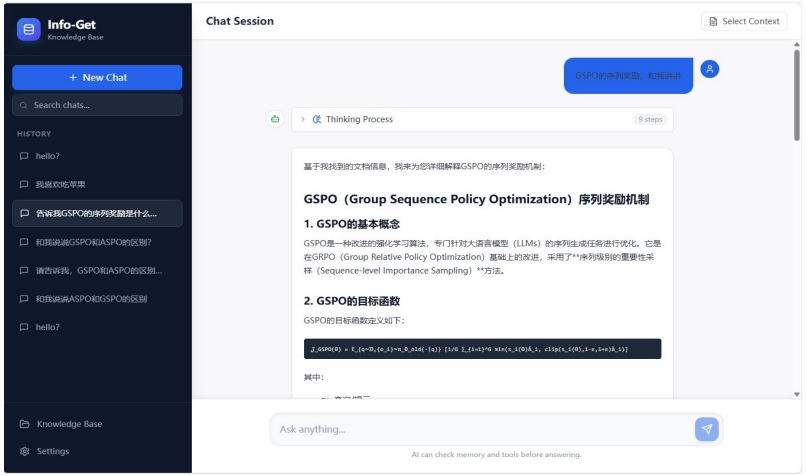
- 技术方案与模块划分
- 受影响的文件路径
- 任务拆解清单

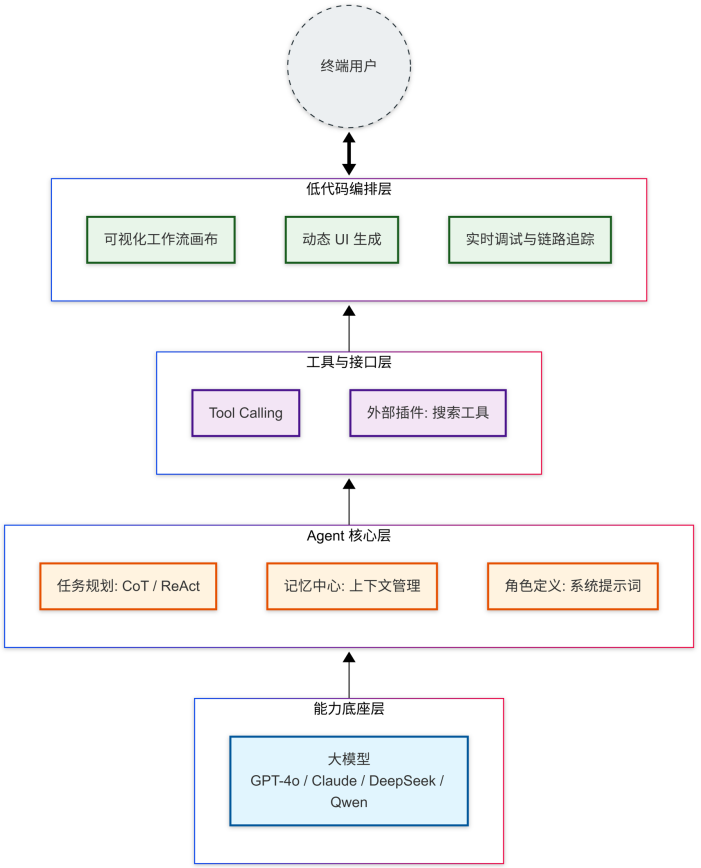
解决“怎么做”

Agentic AI软件开发

融合低代码+大模型+工具编排的闭环智能工作流

- ✓ 智能客服：感知意图 -> 检索知识库 -> 调用下单接口 -> 自动回复
- ✓ 自动化报表：输入指令 -> 自动生成 -> 调取数据库 -> 自动绘图





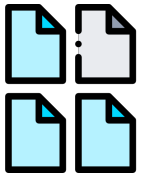
生成式软件开发的潜在问题



- **训练数据平面化**：大模型使用代码数据训练而成，但代码数据所提供的信息是平面化的（缺少软件抽象），仅反映最终结果（缺少决策过程），不足以支撑大模型获得更高层次上的智能化开发能力



- **幻觉问题**：可能生成不存在的API，引用错误/过时的库、参数或配置，导致代码看似合理但无法运行或暗含缺陷

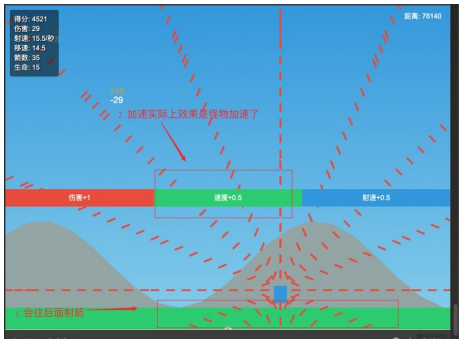


- **长上下文处理能力不足**
  - ✓ 模型能够处理的上下文长度有限，超出范围会被截断导致信息丢失
  - ✓ 即使上下文窗口够长，但是当上下文增长时，模型难以注意到所有信息，容易导致关键信息被忽略或遗漏



生成式应用开发：大模型代码生成

Claude 3.7 Sonnet 射箭小游戏 一次性开发



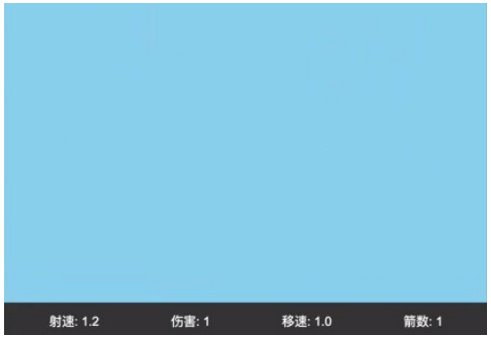
一次性生成 =
大部分功能 + 小部分BUG



迭代修复 = 大部分BUG
+ 小部分功能

Claude 3.7 Sonnet 射箭小游戏 迭代开发



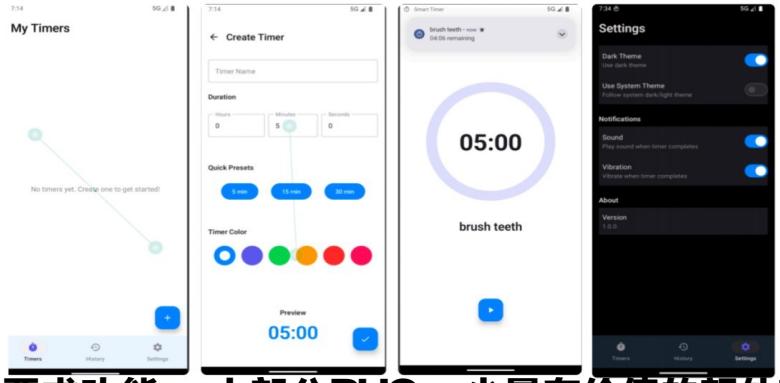


少量需求 = 完善的初始功能 需求增加 = 修BUG的“无底黑洞”

- **大模型的一次性代码生成能力在显著增强**，常见的小规模应用可一次性生成
- **大模型的迭代修复能力依然较差**，在已有代码基础上迭代修改经常“反复横跳”，导致BUG激增
- **反映出的是大模型有限的整体理解和掌控能力**，缺少对软件的高层抽象和整体构思，经常在迭代开发过程中迷失方向

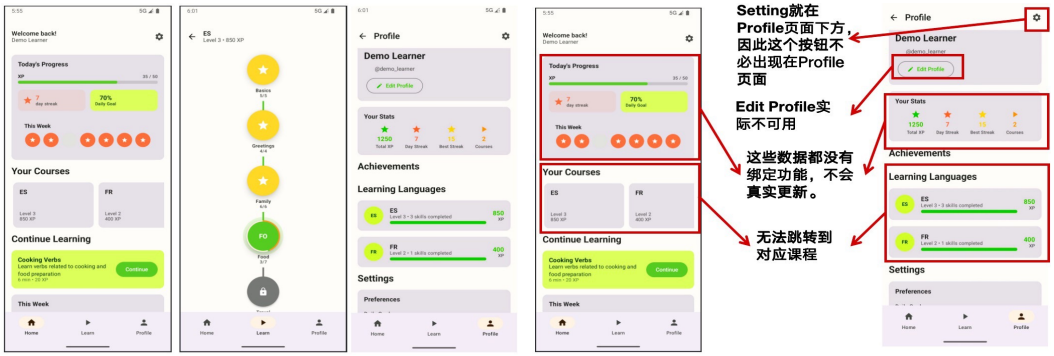
生成式应用开发：大模型Agent

Claude 4 Sonnet + Bolt.new 计时器应用（简单应用）



全部要求功能 + 小部分BUG + 少量有价值的额外功能

Claude 4 Sonnet + Bolt.new 多国语言学习应用（稍复杂）



精致Demo + 大量BUG + 大量遗漏需求 + 大量画蛇添足的功能

- **智能体通过“上下文工程”提升了大模型的编码能力**，使其可以生成更精巧可用的软件
- 然而，**这种提升依然缺乏整体理解和掌控**，导致复杂应用开发上的不足，如大量遗漏需求
  功能间缺少必要的衔接以及**大量“画蛇添足”的功能**
- 亟需构建**高层概念层面上的整体规划和协调能力**，通过需求拆解和设计规划指导软件的增量、迭代开发过程

人机协作演进式软件开发

《ChatGPT协作开发实录：编程新手的试验探索》

CodeWisdom 2024-03-20 08:03 上海

4人听过

点击蓝字 关注我们

张刚老师在《ChatGPT结对编程实录：提升生产力，还是被代替？》一文中，在没有编写任何一行代码的情况下自己指导ChatGPT实现了俄罗斯方块这个游戏。在看完整篇文章后我们深刻感受到了ChatGPT在其中体现的代码生成的能力，于是抱着下面的一些疑问也开始了我们关于ChatGPT结对编程的探索：

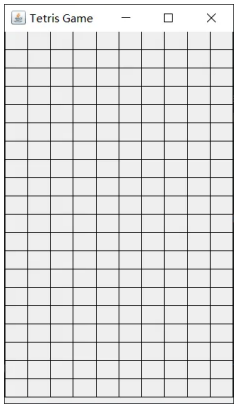
1、如果是对于完全没有开发俄罗斯方块游戏经验的人来说，ChatGPT真的可以提供端到端的开发帮助吗？
2、如果ChatGPT有这么强大的开发能力，那么对于程序员我们如何更好的利用ChatGPT去辅助我们的开发呢？

在这次探索中，我们一共进行了三种试验，这三种试验使用的都是GPT-4，并且全程人工没有写一行代码，只是将GPT生成的代码拷贝到编译器中。

详见CodeWisdom公众号（24年3月）

新手（未开发过俄罗斯方块游戏的研究生）按照不同的思路尝试了三次

第1次试验：逐类实现



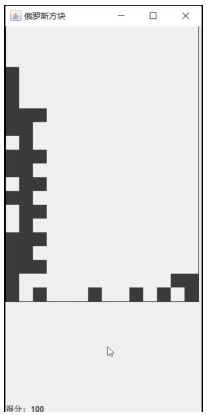
依赖ChatGPT进行分解，然后逐类实现，得到一个完全静止的面板，无法实现游戏功能

第2次试验：逐特性实现



按照自己的理解进行特性分解，实现了一个简易游戏，功能简单且错误多

第3次：按专家思路



按照专家的思路，取得了较好的效果

学生总结：ChatGPT生成代码的能力是有目共睹的，但如何更好地使用ChatGPT，也紧密取决于使用者本身对于开发项目的理解和开发设计规划。在对项目有合理规划，并且对项目整体进行细粒度feature的拆解，甚至按照合理的顺序去逐步实现的情况下，ChatGPT能够发挥出更强的能力。

专家反馈：演化才是本质。这个实验表明， 逐层递进的演进式设计方案更容易得到结果，在俄罗斯方块这个规模的应用尤其明显。合理的演化路径

本章小结

- 生成式软件开发：以大模型为核心引擎，通过自然语言交互、工作流编排和能力集成，快速构建传统软件或智能化软件的新型软件开发范式
- 涉及的关键技术
  - ✓ 提示工程：结构化提示词、示例增强、思维链增强、角色扮演
  - ✓ 上下文工程：实现跨越单次交互的丰富和连续的上下文管理
  - ✓ 智能体（Agent）：以大模型为决策中枢，自主完成任务的智能实体
- 智能化IDE：在传统IDE的基础上原生集成大模型编程助手
- 六个阶段的进阶实践：纯对话式单文件程序、基于自动上下文管理的多文件项目、基于手动上下文管理的多文件项目、规则约束的多文件项目、规约驱动的多文件项目、Agentic AI软件

理解大模型代码生成的不足，强化测试与验证意识和能力

生成式软件开发

End

1. 生成式软件开发概述